Homework 2 - Interest Points and Feature Descriptors

1. **Interest point operator**

I chose to implement the Harris Corner detection because it was the method I felt was best outlined in the slide show presentations. The overall process includes (Slide 34 lec03_features):

1. Compute M matrix for each image window to get their cornerness scores
2. Find points whose surrounding window gave larger corner response (f > threshold)
3. Take the points of local maxima i.e perform non-maximum suppression

Formula for second moment matrix:

$$\mu(\sigma_I,\sigma_D) = g(\sigma_I) * \begin{bmatrix} I_x^2(\sigma_D) & I_x I_y(\sigma_D) \\ I_x I_y(\sigma_D) & I_y^2(\sigma_D) \end{bmatrix}$$

(

Formula for cornerness function:

$$har = \det[\mu(\sigma_I,\sigma_D)] - \alpha[\text{trace}(\mu(\sigma_I,\sigma_D))^2] =$$
$$g(I_x^2)g(I_y^2) - [g(I_x I_y)]^2 - \alpha[g(I_x^2) + g(I_y^2)]^2$$

Formula for Harris response:

$$R = \det(M) - \alpha\,\text{trace}(M)^2 = \lambda_1\lambda_2 - \alpha(\lambda_1 + \lambda_2)^2$$

High level pseudocode

```
Def find_interst_points(image, max_points, scale, mask):

        Ix, Iy = compute_gradiants(image, scale)
        Horz_gradient, Vert_gradient, Between_gradiant =
                        compute_second_moment_matrix(Ix, Iy, Scale)

        R = harris response(Horz_gradient, Vert_gradient, Between_gradiant)

        Non_max = nonmax_suppression(R, radius)

        Sort non_max
```

Clip top max_points of sorted non_max

Return xs, yx, scores

2. Feature Descriptor

- For the implementation of feature Descriptor, I implemented the SIFT vector formation as specified by the documentation. (Slides starting 57 SIFT vector formation)

The goal of SIFT, scale invariant feature transform, is to describe a local area in an image through a feature vector:
The overall process was:

1. Take an image with xs and ys
2. Compute image gradients of magnitude and orientation vectors
3. Divide neighbor grid into 3x3 grid
4. For each cell collect gradients into histogram
    a. Weight votes by gradient magnitude
    b. Incorporate gaussian filter for smoothness
5. Combine histograms into a vector + normalize

Pseudocode:

```
Function extract_features(image, xs, ys, scale):
        Compute image gradients
        For each interest point:
                Find neighbor patch
                Divide into 3x3 spatial grid
                For each pixel in path
                        Compute gradient magnitude + direction
                        Vote into orientation histograms
                Combine histograms into feature vector
                Normalize feature vector
        Return all feature vectors
```

Resources I used to understand the process:
https://www.youtube.com/watch?v=IBcsS8_gPzE&t=24s

3. Matching features

I used the nearest neighbor matching as specified on Slide 72 of lec03 features

NN1 / NN2 is a score where NN1 is the distance to the first nearest neighbor and NN2 is the distance to the second nearest neighbor
This works because we want to identify unique features that aren't repeated throughout the image.
If the top two feature matches result in a small ratio (small numerator large denominator) it would be a good feature to record as there would exist one occurrence of this unique feature to find
If the top two features matches are very similar then the there would occur multiple of the same patterns on the image therefore could provide error when choosing the key location

1. Find closest 2 distances (NN1) and (NN2)
2. Score NN1 / NN2

Function match_features(feats0, feats1, scores0, scores1):
    For each descriptor f in feats0:
        Compute distances from f to every descriptor in feat 1
        Nn1 = smallest distance
        Nn2 = second smallest distance

        Save index
        Score = nn1 / nn2
        Save match and score

4. Hough Transform
Using Hough transform, method outlined in slideshows, in order to find the most optimal translation.

$$\begin{bmatrix} x_i^B \\ y_i^B \end{bmatrix} = \begin{bmatrix} x_i^A \\ y_i^A \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

Tx = xiB - xiA
Ty = yiB - yiA

Overall process:

- Initialize a grid of parameter values
- Each matched pair casts a vote for consistent values
- Find the parameters with the most votes
- Solve using least squares with inliers

Pseudocode:

Function hough_votes(xs0, ys0, xs1, ys1, matches, scores):

Make bank of possible translations

For each feature i in imageA:
Get match
Translate
Cast a vote for translation bin
Find translation with max number of votes

Get inliers set
Use robust least squares to estimate using inliers

5. Object Detection

I chose to implement option two of repeat a single scaled detection over multiple image scales by resizing, so I could make use of the helper methods previously created. The method itself combines the methods created above into one big object detection process.

1. Extract features only from template foreground
2. Extract features from the test images
3. Match template features with test features
4. Use hough voting to predict object center
5. Convert center to bounding box
6. Keep the best scoring detection does

Function object_detection (template_images, template_masks, test img):

For each scale requirement
Resize image to scale
Extract interest points + descriptors
For each image T and mask M:
Extract interest points
Extract descriptors
Match template and test descriptors
Use hough voting to estimate translation
Identify object center
Compute bounding box
Compute confidence score (hough votes)
If confidence score > best_score:
Best_score = confidence
Save bounding box

Return best bounding box

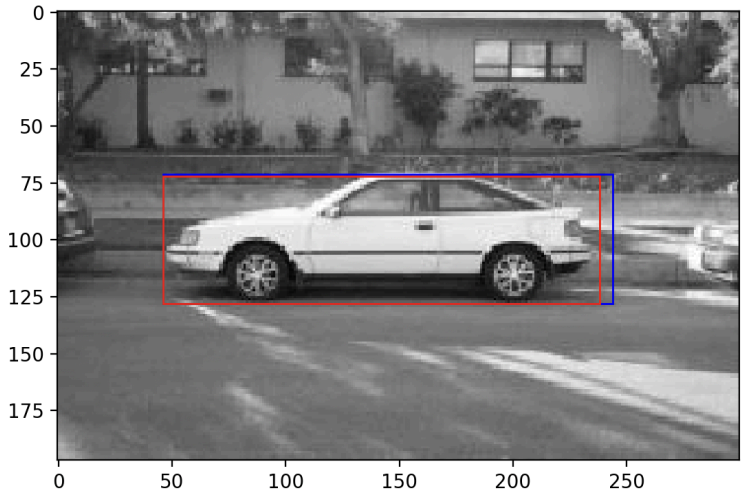
Measuring Runtime + accuracy

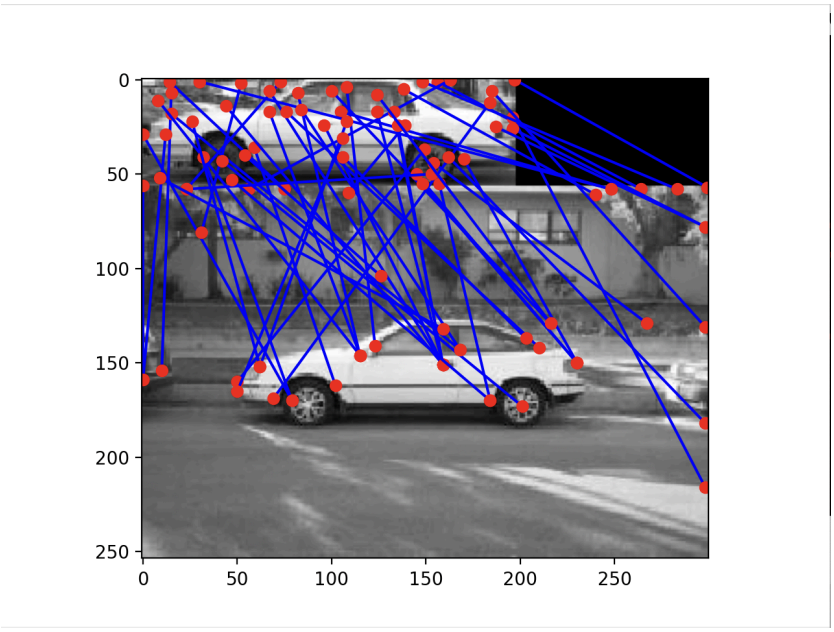
Scales = [0.5, 1.0, 2.0]

[object_detection] best_inliers=4, best_scale=0.5, best_template=3
object_detection runtime: 11.8824 seconds

Scales = [1]

[object_detection] best_inliers=4, best_scale=1.0, best_template=0
object_detection runtime: 2.9765 seconds




0
50
100
150
200
250
0
50
100
150
200
250

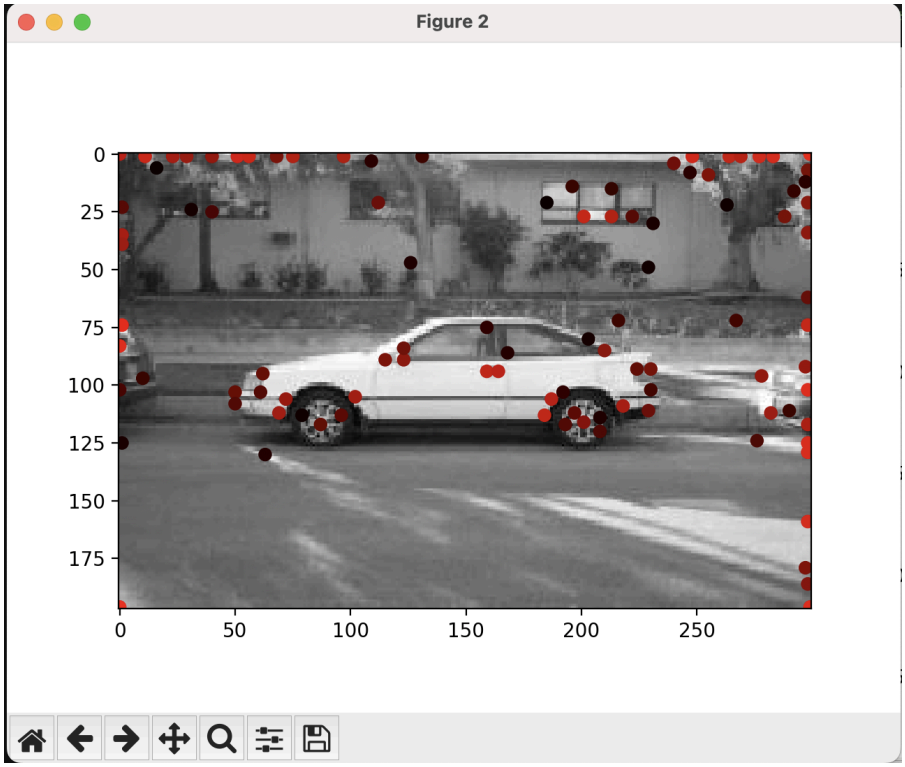


Figure 2
0
25
50
75
100
125
150
175
0
50
100
150
200
250

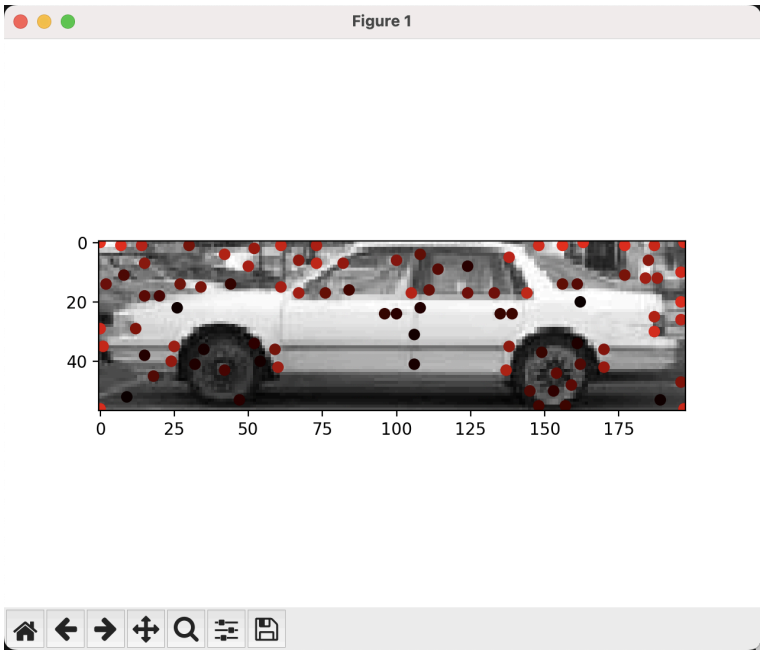


Figure 1
0
20
40
0
25
50
75
100
125
150
175